EVFLOW ENGINEERING

Data Cleaning, Wrangling, and Analytics Report

Project report sections 2.2 and 2.5

Report purpose

This report explains how EVFlow prepares charging-station and vehicle-reference data for analysis. It follows a project-report format and separates observed results, derived attributes, scenario estimates, and capabilities that are not yet implemented.

Load	Filter	Standardize	Enrich	Merge	Load
raw snapshots	invalid records	common columns	derived fields	75 m matching	PostGIS

3,569	2,931	17.9%	60
normalized station records	candidate station clusters	record reduction	EV catalogue models

Document	Data Cleaning, Wrangling, and Analytics Report
Sections	2.2 Data Preparation and 2.5 Data Analytics
Version	1.0
Prepared	24 July 2026
Owner	EVFlow Data Engineering

All station figures in this report refer to the current national API seed. The separate Jakarta notebook filters to a study boundary before clustering and produces different counts.

2.2 Data Preparation

2.2.1 Data Cleaning and Wrangling

Charging Station Dataset

The station dataset combines the cached PLN SPKLU, Open Charge Map, and OpenStreetMap snapshots. Cleaning is performed after loading so the original raw files remain unchanged and can be used to reproduce or audit the transformation.

- Data Loading: We loaded 3,029 PLN records from _petaspklu_all.json, 527 Open Charge Map records from ocm_jakarta.json, and 13 OpenStreetMap elements from osm_charging_jakarta.json. These 3,569 inputs are source observations, not confirmed unique physical stations.
- Filtering: PLN records with missing, non-numeric, non-finite, or zero coordinates were excluded. OCM and OSM records were required to contain both latitude and longitude. A station with incomplete descriptive information was retained when its location remained usable.
- Column Mapping: Source-specific fields were mapped into one station structure. For example, PLN nama_lokasi, OCM AddressInfo.Title, and the OSM name or operator tags all map to the canonical name field.
- Column Selection: We retained the fields needed by the application and analysis: source ID, source name, coordinates, station name, address, province, city, operator, power, charging type, connector observations, status, and verification date.
- Data Type Conversion: Coordinates and power values were converted to numeric types. Text such as '150 kW' was reduced to the numeric value 150. OCM connection quantities and OSM capacity tags were converted to counts where values were available.
- Missing Values: Blank optional fields were represented as null rather than replaced with invented values. NaN power values were converted to null. Missing names or addresses could later be filled from another source when two records were merged.

Cleaning rule

Location is mandatory because spatial search and record matching depend on it. Name, address, operator, and connector details are useful but may remain unknown without removing the station observation.

Connector and Charging-Speed Wrangling

- Charge-Type Normalisation: Charging labels were trimmed and converted to lowercase. Equivalent values such as 'ultrafast' and 'ultra fast' were standardised to ultra_fast.
- Connector Enrichment: Reliable connector-standard values were not consistently available. Power above 22 kW, or a fast/DC hint, was classified as inferred CCS2. Power at or below 22 kW, or a slow/medium/AC hint, was classified as inferred AC Type 2. Unknown evidence produced no connector classification.
- Speed Classification: Power values were grouped as slow at 7 kW or below, medium above 7 to 50 kW, fast above 50 to 150 kW, and ultra-fast above 150 kW. The inference flag was preserved so these values are not presented as verified equipment specifications.

2.2.2 Data Merging and Integration

- Record Ordering: Normalized records were sorted by source priority: PLN first, followed by OCM and OSM. Sorting by source ID within each group makes the result repeatable for the same input files.
- Spatial Matching: For each record, we calculated Haversine distance to existing cluster anchors. A record was merged into the first anchor within 75 metres. If no anchor met the threshold, the record created a new candidate cluster.
- Field Consolidation: The anchor retained its ID and coordinates. Populated higher-priority fields were preserved, while missing name, address, region, operator, status, or verification values could be filled from lower-priority records. Every contributing source was retained in sources[].
- Connector Consolidation: Connector observations were grouped by connector type and power. When two sources reported the same configuration, the maximum connector count was retained instead of adding the counts and risking double-counting the same hardware.
- Data Loading: The resulting station reference data was loaded into PostgreSQL/PostGIS. Coordinates were stored as WGS84 point geometry. The seed replaces the existing station table in one batch; it is not an incremental update process.

Processing result

The current national seed reduces 3,569 normalized source records to 2,931 candidate station clusters: 638 fewer records, equivalent to a 17.9 percent reduction. The clusters are probable physical sites, not independently field-verified stations.

EV Model Catalogue

Vehicle specifications are prepared separately from station records. The catalogue supports range-aware routing and is not merged into the station table.

- Data Loading: We loaded the 60-row Indonesian EV catalogue from the extracted Kaggle CSV. If the configured CSV is unavailable, the loader can read the CSV member from the committed ZIP file.
- Filtering: Rows without a Vehicle Name were excluded because a stable model identity could not be constructed. Other incomplete attributes were retained as null when the model name remained valid.
- Column Wrangling: Vehicle names were trimmed and converted into lowercase slug IDs. The first name token was treated as the make and the remaining tokens as the model name.
- Data Type Conversion: Numeric values were extracted from free-text battery and range fields. Decimal commas were converted to decimal points, allowing strings such as '26,7 kWh' to become the numeric value 26.7.
- Range Standardisation: When a range field contained multiple values, such as '200 - 300 km', the lower value was retained. This conservative choice reduces the risk of overstating whether a vehicle can reach a station.
- Duplicate Removal: Rows that produced the same model slug were consolidated by retaining the first parsed record. The resulting catalogue is cached in memory after its first successful load.
- Data Output: The catalogue supplies model ID, make, model, battery capacity, driving range, price range, charging time, and source URL to the EV model API and range calculation.

Method distinction

Parsing numbers and removing duplicate slugs are cleaning and wrangling operations. Multiplying model range by state of charge and a 0.85 safety factor is a later decision-support calculation, not part of cleaning.

Data Quality Considerations

The current cleaning code does not perform complete geographic-bound checks for OCM and OSM, schema-drift detection, or automated review of ambiguous 75 metre matches. Connector standards are inferred and must remain labelled as estimates. These constraints are carried into the analytics interpretation below.

2.5 Data Analytics

EVFlow currently supports descriptive analysis and operational decision support. It does not contain a trained demand-forecasting model or a consistent historical utilization series. Foresight results are therefore scenario estimates rather than predictions.

2.5.1 Hindsight Insights

- Data Consolidation: The three station sources contributed 3,569 normalized records. Spatial record linkage consolidated these observations into 2,931 candidate clusters, showing that a single-source count would overstate the number of distinct sites represented by the combined dataset.
- Source Overlap: The merged output contains 2,413 PLN-only clusters, 355 PLN-and-OCM clusters, 150 OCM-only clusters, seven clusters represented by PLN, OCM, and OSM, and six OSM-only clusters. This reveals where community sources confirm or extend the PLN snapshot.

Source presence	Clusters	Interpretation
PLN only	2,413	Represented only by the PLN snapshot
PLN + OCM	355	Cross-source confirmation within the 75 m rule
OCM only	150	Community records not merged with PLN
PLN + OCM + OSM	7	Represented by all three station sources
OSM only	6	Independent OSM records outside another cluster

- Geographic Distribution: The largest recorded provincial groups are DKI Jakarta with 696 clusters, West Java with 522, and Banten with 280. PLN is nationwide while OCM and OSM were collected for Greater Jakarta, so provincial source coverage is not uniform.
- Data Completeness: Power is available or derivable for 2,925 of the 2,931 clusters, while 2,889 contain an address. High field coverage does not establish that values are current or independently verified.

Derived charging-speed distribution

Counts describe the current merged snapshot; speed tier is derived from connector power.

- Operational History: Charging sessions and wallet top-ups store timestamps, status, energy, and monetary values for authenticated users. These records can support retrospective analysis after sufficient real usage accumulates, but the current API exposes user-scoped history rather than a system-wide analytics warehouse.

2.5.2 Foresight Insights

The implemented foresight logic supports immediate travel decisions. It estimates reachability under stated assumptions; it does not forecast long-term charger demand or network growth.

Battery reachability calculation

Usable range is calculated as:

```
usable range = model range x (state of charge / 100) x 0.85
```

- Battery Reachability: The calculated usable range is compared with road distance to the nearest route candidate. The API returns a within-range flag so the driver can distinguish a nearby station from one that may be beyond the estimated remaining range.
- Route Scenarios: The routing backend can compare paths weighted by distance or travel time using a prepared GraphML road network. Travel time is based on graph attributes or fallback speeds and does not include live congestion, closures, weather, elevation, or battery degradation.
- Coverage Scenarios: Station density, nearby-search results, and route distance can identify areas that warrant further investigation. They cannot by themselves predict charging demand because the source snapshots do not contain utilization, queue, traffic, or local EV-population time series.
- Trend Limitation: The station sources are point-in-time snapshots and are not temporally aligned. The current dataset therefore cannot support defensible claims about historical growth, future shortages, or operator expansion trends.

Analytics Required for Forecasting

Required dataset	Why it is needed
Timestamped connector availability	Utilization rates, downtime, queues, and peak-period behavior
Verified charging sessions	Delivered energy, session duration, completion, and failed-charge demand
Traffic and trip patterns	Expected route demand rather than station proximity alone
Regional EV population	A denominator for demand and infrastructure adequacy
Repeated station snapshots	Historical additions, removals, status changes, and data freshness

Interpretation rule

Use the terms estimate, scenario, and within-range assessment for current foresight outputs. Do not describe them as machine-learning predictions or demand forecasts.

2.5.3 Actionable Insights

- Location-Based Recommendations: Drivers can identify nearby stations using PostGIS distance search and then narrow the result by connector type, charging speed, power, province, city, or map boundary.
- Route and Battery Guidance: When the road graph is available, EVFlow can calculate route distance and duration and indicate whether the selected EV model appears able to reach the nearest route candidate at its current state of charge.
- Source Verification Priority: OCM-only, OSM-only, conflicting, and near-threshold clusters should be reviewed first. These records have a higher likelihood of incomplete details or unresolved duplication.
- Connector Verification: High-power, multi-connector, and commercially important sites should be checked against equipment-level evidence. Inferred AC Type 2 and CCS2 values remain provisional until verified.
- Geographic Comparability: A common study boundary must be applied before comparing source coverage or recommending regional priorities. Nationwide PLN records cannot be compared directly with the Greater Jakarta OCM and OSM extracts.
- Merge-Threshold Review: The clustering result should be compared at 25, 50, 75, and 100 metres. Ambiguous matches in malls, rest areas, or dense urban sites should be inspected manually before publication.
- Refresh Control: Every refresh should preserve a dated raw snapshot and record the source URL or query, retrieval time, licence, row count, checksum, and transformation-code revision. Quality metrics should be recalculated before release.

Current Implementation Boundary

Status	Capability
Implemented	Station aggregates, province/city counts, spatial search, connector and speed filters, EV range calculation, and user-scoped transaction histories
Conditional or partial	Backend road routing requires an external GraphML file; the frontend route planner remains a placeholder; most business-planning KPIs are static
Not implemented	Demand forecasting, ML prediction, system-wide utilization analytics, automated periodic source refresh, and historical station-trend analysis

Reporting position

EVFlow can describe the present station snapshot and support route and battery scenarios. Historical trend analysis and predictive infrastructure planning require new time-series data and analytics pipelines before they should be presented as product capabilities.

Implementation references

Source normalization: backend-ev-flow/api/sources.py | Connector rules: backend-ev-flow/api/connectors.py | Spatial merging: backend-ev-flow/api/dedup.py | PostGIS seed: backend-ev-flow/scripts/seed_db.py | EV catalogue: backend-ev-flow/api/evmodels.py | Aggregates and routing: backend-ev-flow/api/stations_repo.py and backend-ev-flow/api/routing.py